PayGuard Escrow: 2-Member Team Work Split

Comprehensive Technical Responsibilities, Shared Contracts, Timeline & Integration Checkpoints

1. High-Level Role Division & Mindsets

Dimension	Member A: Platform, Payments & Frontend	Member B: AI Agent & Verification Engine
Core Focus	Everything the user sees, plus the money & auth	Everything that tests the site and decides the score
Riskiest Element	PayPal authorize/capture flow and webhooks	Reliable Kernel browser agent & LLM test execution
Team Mindset	"Money is correct and the UI is smooth"	"Results are consistent and explainable"
Code Ownership	/app, /components, /lib/paypal	/worker, /lib/agent, /lib/scoring

2. Member A: Detailed Responsibilities

Member A is responsible for platform infrastructure, transactional database models, security/auth, PayPal sandbox integrations, and the full frontend user experience.

Area	Detailed Tasks & Scope
Project Setup	Repo setup (Next.js + TS + Tailwind + shadcn/ui), Supabase initialization (DB, Auth, Realtime, Storage), Vercel deployment setup, Sentry monitoring, CI linting workflows.
Database & Schema	Create and execute migrations for all 12 core entities: User, Job, Checklist, ChecklistItem, PaymentAuthorization, Submission, VerificationRun, TestResult, PaymentEvent, Report, Dispute, AuditLog.
Auth & Jobs	User sign up / OAuth login, job creation flow, client/freelancer dashboard, email invite link generator, role-based access control (RBAC) middleware.
State Machine	Job state machine module with strict state transitions (DRAFT -> AWAITING_FUNDING -> FUNDED -> VERIFYING -> PAID / PARTIAL / HELD -> DISPUTED / EXPIRED) and unit tests.
Checklist UI	Interactive review and edit UI, dual approval controls (client & freelancer), immutable versioning and locking mechanism.
PayPal Engine	Complete integration: Order creation (intent=AUTHORIZE), Authorization, Capture (100% / 70%), Void, Re-authorization, Webhook signature verification and deduplication, Authorization Watcher cron.
Frontend UI/UX	Milestone stepper timeline (live color transitions), Submit Work interface, Live Verification screen (Kernel embed + activity feed), Decision Report view, evidence lightbox, responsive & WCAG 2.1 AA accessibility pass.
Notifications & Deploy	Transactional emails via Resend, Vercel web app production configuration, webhook URL registration, and smoke testing.

Member A Code Interface Contract (lib/paypal/index.ts):

```
// Owned by Member A, called by Member B's settlement orchestrator
export async function captureAuthorization({
jobId, runId, amountCents, finalCapture
}): Promise<PaymentResult>;

export async function voidAuthorization({ jobId, reason }): Promise<PaymentResult>;
export async function getAuthorizationStatus(jobId): Promise<AuthStatus>;
// Requirement: Must be idempotent using key 'job:{id}:run:{id}:capture',
// writing PaymentEvent row & updating Job state in a single DB transaction.
```

3. Member B: Detailed Responsibilities

Member B is responsible for cloud browser execution, AI test generation, worker queue orchestration, deterministic scoring, and settlement execution.

Area	Detailed Tasks & Scope
Demo Target Sites	Build and host test target sites: a fully working site (100%), a mobile-layout broken site (70%), a login-broken site (<60%), a console-error site, and a prompt-injection attack test page.
Checklist Generator	LLM prompt design with JSON schema output, server-side schema validation (3–12 items, weights sum to 100, console health item present, critical flag handling).
Worker & Queue	Background worker service setup on Railway/Fly using BullMQ + Redis (or Postgres queue) for asynchronous verification runs.
Browser Agent	Kernel cloud browser session creation, Playwright CDP integration, browser primitives (goto, viewport resize, console/network error capture, element presence, overflow detection), LLM-assisted login with 15-action cap, 90s/8min timeouts, and 3-run majority voting.
Evidence & Replay	Screenshot uploading to storage, terminal log excerpt extraction, session replay / live-view stream URL generation.
Scoring Engine	Pure deterministic scoring function: sum of weights of passed items, critical-fail override (score=0), boundary unit tests (59, 60, 89, 90).
Settlement Orchestrator	Derives payout percentage from score tier, invokes Member A's PayPal capture/void functions with idempotency keys, handles retries and error reporting.
Report Writer & Security	LLM report writer generating plain-English rationale, SSRF URL validator (blocking loopback/private IPs), and prompt-injection defense verification.

Member B Event Stream & Output Interface Contract:

```
// Realtime event stream emitted during verification run (consumed by A's UI):
type RunEvent = { runId: string, type: 'step' | 'test_result' | 'viewport' | 'score' | 'payout', message:
string, data?: any, ts: number };

// Final run result payload handed from B to A:
type RunResult = {
runId: string, score: number,
tier: 'PAID' | 'PARTIALLY_PAID' | 'HELD' | 'NEEDS_REVIEW',
payoutPercent: number, results: TestResult[], reportId: string
};
```

4. Shared Team Contracts (First 1–2 Hours)

To ensure zero friction during independent parallel development, both members must establish and agree upon these 6 core contracts prior to coding:

- **1. Database Schema:** Member A drafts initial SQL/Prisma migration for all entities; Member B reviews and approves schema before database creation.
- **2. TypeScript Shared Types:** Commit shared definitions in `/types` for `ChecklistItem`, `TestResult`, `RunResult`, `JobStatus`, and `RunEvent`.
- **3. Activity Feed Event Protocol:** Exact JSON schema and message types emitted to Supabase Realtime during live verification.
- **4. Module Function Signatures:** Exact stub implementations for `captureAuthorization`, `voidAuthorization`, and `triggerVerification`.
- **5. Environment Variable Registry:** Unified `.env.example` containing `PAYPAL_*`, `KERNEL_API_KEY`, `LLM_API_KEY`, and `REDIS_URL`.
- **6. Git Branching Policy:** Protected `main` branch, short-lived feature branches, and mandatory peer review before merging.

5. 4-Day Development Timeline & Sync Milestones

Day	Member A (Platform & UI)	Member B (Agent & Scoring)	Sync & Integration Goal
Day 0 (1–2h)	Database schema & TypeScript type stubs	Review schema & worker skeleton	Agree on shared contracts & API interfaces
Day 1	Auth, job CRUD, dashboard, state machine	Demo target sites, Kernel connection, Playwright primitives	Checkpoint C1: Job creation + local agent run
Day 2	PayPal order/authorize, webhooks, FUNDED state, blue timeline	LLM login test, test runner with voting, checklist generator	Checkpoint C2/C3: Checklist generator connected to UI
Day 3	Capture/void calls, Submit page, Live Verification UI, Report view	Scoring engine, settlement orchestrator, report writer, storage	Checkpoint C4/C5: Full end-to-end run & 3-tier payouts
Day 4	Polishing, error states, mobile audit, watcher cron, deploy	SSRF & prompt injection tests, reproducibility verification	Feature freeze, 5x rehearsal, backup video recording

6. Integration Checkpoints & Verification

ID	When	Integration Verification Check	Verifier
C1	End Day 1	Agent executes against local demo site; UI creates jobs successfully	Both Members
C2	Mid Day 2	Checklist generated by LLM appears and is editable/approvable in UI	Both Members
C3	End Day 2	Job reaches FUNDED state via PayPal sandbox authorization	Member A
C4	Mid Day 3	Verification run triggered from UI streams live step events to screen	Both Members
C5	End Day 3	All 3 payout tiers (100%, 70%, 0%) execute; double-capture prevented	Both Members
C6	Day 4	Full end-to-end demo completes 5 consecutive times without failure	Both Members

7. Demo Day Roles & Collaboration Rules

Demo Day Role Allocation:
• Member A (Presenter & Pitcher): Leads the presentation, introduces problem, pitches 'AI Escrow & Autonomous Money', drives user narrative.
• Member B (Technical Operator): Triggers live URL submission, monitors Kernel browser session logs, manages fallback demo video.
• Q&A Division: Member A handles UX, business model, and PayPal compliance questions. Member B handles LLM reliability, Playwright agent mechanics, and security controls.

- Daily 10-Minute Standup: Sync every morning to highlight blockers, code dependencies, and current milestone targets.
- Zero Blocker Policy: Never wait for unbuilt components — utilize mocked JSON data and stubbed function responses.
- Double-Reviewed Financial Code: Every change to PayPal capture logic, void handling, or scoring rules requires code review from both members.
- P0/P1 Bug Triage: On Day 4, resolve only P0 (demo breaking / payment incorrect) and P1 (visible UI flaw) issues.
- Workload Rebalancing Protocol: Since Member B's agent is on the critical path, Member A will absorb report page UI / email tasks if Member B is delayed.